



به نام خدا
دانشگاه تهران
دانشکده فنی
دانشکده برق و کامپیوتر



گزارش فاز هفتم تمرین وریفیکیشن HDL

نام و نام خانوادگی: بهزاد جنتی

شماره دانشجویی: 810103098

استاد: دکتر سیامک محمدی

ارشد گرایش مهندسی معماری کامپیوتر

بنام خداوند جان و

در این فاز از پروژه وریفیکیشن قرار است تا یک اسرشن های ltl و ctl مدل داده شده (FIFO) رو به کمک نرم افزار NuSMV و در بستر لینوکس پیاده کنیم (Model Checking).

2 - Description of provided FIFO model

A block diagram of the FIFO circuit is shown in Figure 1.

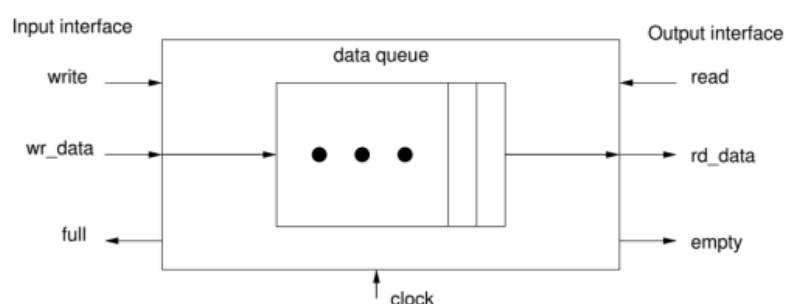


Figure 1. FIFO block diagram

دیزاین داده شده به ما یک صف FIFO است که یک input interface و یک output interface دارد که در بخش اول در صف مینویسیم و در بخش دوم از صف میخوانیم.

دو سیگنال full و empty در تصویر وجود دارند که سیگنال full زمانی یک میشود که صف ما پر شده باشد و سیگنال empty در ابتدای کار که صف خالی و چیزی در آن نوشته نشده است مقدار یک دارد.

اولین بخش سوال از ما میخواهد که طبق توضیحات داده شده در فایل تمرین اسرشن های ltl رو بنویسیم.

(2) صورت اولین اسرشن به شکل زیر است:

(a) *It is never the case that the FIFO indicates simultaneously it is both empty and full.*

این property بیان میکند که هیچ وقت دو سیگنال full و empty نمیتوانند با هم همزمان مقدار یک داشته باشند.

شکل نوشتن این property LTL رو در شکل زیر مشاهده میکنید:

نکته) دقت شود تمام property های خواسته شده در فایل fifo_properties.smv و در بخش مشخص شده به هر کدام نوشته شده است.

```
fifo-properties.smv X Workspace Trust
C: > Users > Asus > Downloads > fifo-properties.smv
24
25 --2: (a)
26 LTLSPEC G !(empty & full)
27
28
```

عبارت بالا بیان میکند که همان طور که قبلا گفته شد در هیچ کدام از استیت ها نباید شرط رو به رو برقرار باشد. (در اینجا LTL است پس به ازای همه ی مسیر ها باید برقرار باشد) و G هم خلاصه global است و نشان میدهد شرط رو به رو در تمام لحظات زمانی (همه حالات سیستم) برقرار است.

3) صورت دومین property به صورت زیر است:

(b) If write is asserted forever and read is never asserted, then the FIFO eventually becomes full.

این گزاره بیان میکند اگر برای همیشه در صف بنویسیم و از آن نخوانیم بالاخره صف مون پر میشود و سیگنال full=1 میشود.

```
fifo-properties.smv X Workspace Trust
C: > Users > Asus > Downloads > fifo-properties.smv
28
29 --3: (b)
30 LTLSPEC G (write -> F !empty)
31
32
```

عبارت بالا میگه همیشه شرط اگه write داشته باشیم بالاخره (finally) سیگنال empty برابر صفر خواهد شد به عبارتی در نهایت full=1 خواهد شد. در چند بهتر بود به شکل زیر مینوشتیم:

LTLSPEC G (write & !read) -> F full

نکته) علامت G همان always و F معادل finally میباشد.

4) صورت گزاره بعدی به صورت زیر است:

(c) *At any time, if a 1 is presented to the FIFO data input and write is asserted, then eventually a 1 will appear on the FIFO data output.*

With further reasonable assumptions added after the *if* concerning FIFO signals such as read, empty or full, to ensure the property checks true. Consult the NuSMV user guide for information on *word constants* to be able to refer to the number 1.

گزاره بالا داره میگه اگه ورودی عدد ثابت 1 در ورودی صف قرار داشته باشه و سیگنال write هم یک باشه (داده در صف نوشته شود) پس بالاخره عدد 1 در خروجی صف ظاهر خواهد شد.

که این هم گزاره درست و منطقی است.

```
33 --4: (c)
34 LTLSPEC G (wr_data = 0ud1_1 & write & !full -> (G(read) -> F rd_data = 0ud1_1))
35
```

شکل بالا هم property نوشته شده LTL است که از روی گزاره گفته شده نوشته شده است.

قسمت اول داره میگه اگه دیتای ورودی (برای نوشتن) برابر با مقدار ثابت 1 (0u1_1) باشد و سیگنال write=1 و full=0 باشد (یعنی صف جا داشته باشد که در آن بنویسیم) در نتیجه همیشه اگر سیگنال read=1 باشد بالاخره داده ثابت 1 رو از صف میخوانیم.

5) صورت گزاره 5 به شکل زیر است:

(d) The same property as in (c), except that it is phrased to hold for any data value, not just the value 1. Use the 'frozen variable' data1 to do this. Consult the NuSMV user guide for documentation on *frozen variables* (also sometimes known in temporal logic as *rigid variables*).

مثل گزاره قبل است منتها برای همه مقادیر این گزاره درست است نه فقط مقدار ثابت 1 ، همچنین در صورت گزاره گفته از Data1 به عنوان frozen variable استفاده کنید.

```
37 --5: (d)
38 LTLSPEC G (wr_data = data1 & write & !full -> (G(read) -> F (rd_data = data1)))
39
40
```

همان طور که میبینید دقیقا مثل گزاره قبل است با این فرق که دیگر به جای عدد ثابت 1 از متغیر `data1` استفاده شده است.

به سادگی قابل تشخیص است که این گزاره هم درست است.

(6) صورت گزاره به شکل زیر است:

(e) the same property as in (d), except that, in addition, it requires the empty output of the FIFO to be set to false at all times *inbetween* the time the write of the data is set up and the time the data can first be read out, but not actually at either of these times. You may take advantage of the fact that the earliest we expect the data to appear is the step after it is written.

این گزاره مثل گزاره های قبل است با این فرق که عنوان میکند سیگنال `empty` از زمانی که داده در صف نوشته شده است تا زمانی که از صف خوانده میشود باید `false` (همون صفر) بماند.

در واقع وقتی چیزی توی صف نوشتیم در سریع ترین زمان ممکن در سیکل بعد میتواند در خروجی ظاهر شود و مشخصا در این تایم صف خالی نیست چون یه چیزی توش نوشتیم و این مورد هم بدیهی است که درست باشد.

```
41 --6: (e)
42 LTLSPEC G ( !full & wr_data = data1 & write -> X(G(read) -> !empty U read))
43
```

`property` بالا عنوان میکند که همیشه اگر صف پر نباشد و داده ای برای نوشتن باید و سیگنال `write` هم یک باشد (لحظه نوشته شدن داده در صف) در نتیجه در سیکل بعد (`X=next`) همیشه `read=1` است و `empty=0` تا وقتی که دیگه چیزی برای خواندن باشد.

این مورد هم مشخص است که درست است.

(7) صورت این گزاره به شکل زیر است:

(f) a similar property to that for (d), except that it assumes that two possibly-distinct data values are input on consecutive steps, and checks for the same two values appearing on the output on consecutive steps. Use the provided frozen variables `data1` and `data2` to refer to the two data values.

مثال گزاره قبل هست با این فرق که درباره دو ورودی پشت سر هم صحبت میکند و شرط رو بررسی میکنه که این مورد هم مشخصا درست است.

```
--7: (f)
--LTLSPEC
LTLSPEC G ((write & (wr_data = data1)) & (X (write & (wr_data = data2))) -> (X (rd_data = data1) & X (rd_data == data2)))
```

حال به سراغ گزاره های ctl میرویم:

(1) گزاره اول به شکل زیر است:

(a) *There exists a run for which, at some time onwards, the FIFO is always full,*

این گزاره به وضوح اشتباه است چون بالاخره عملیات خواندن انجام میشود و اون **always** که گذاشته کار رو خراب میکند.

نکته مهم) **property** های CTL برای بعضی مسیر ها بررسی میشود و مثل LTL نیست که همه مسیر ها حتما باید شرط گفته را داشته باشند (برقرار باشد).

```
55 -- 0: (a)
56 CTLSPEC EF EG full
57
```

(2) گزاره دوم به شکل زیر است:

(b) *From every reachable state in which the FIFO is full, there exists a path along which the FIFO eventually becomes empty.*

بله این گزاره کاملا درست است.

چون گفته از همه ی استیت هایی که در آنها صف پر است یک مسیر وجود دارد که به صف خالی برسد.

```
57 --1: (b)
58 CTLSPEC AG [(full -> EF (empty))]
59
60
```

این مورد در فایل نهایی ارسال شده اصلاح شده است.

حال باید در این قسمت از طریق وارد کردن دستور `NuSMV -pre cpp fifo.smv` در ترمینال در لینوکس این `property` ها رو در مدل بررسی کرده و خروجی مشخص شود (یا `pass/fail`) که برای `fail` یک مثال نقض نیز ارائه میدهد. -> روش `model checking`

خروجی نهایی اجرا در لینوکس:

```
-- specification EF (AG full) is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
  data1 = 0ud1_0
  data2 = 0ud1_0
  buffer[0] = 0ud1_0
  buffer[1] = 0ud1_0
  buffer[2] = 0ud1_0
  buffer[3] = 0ud1_0
  buffer[4] = 0ud1_0
  rd_p = 0
  wr_p = 0
  empty = TRUE
  write = FALSE
  wr_data = 0ud1_0
  read = FALSE
  rd_data = 0ud1_0
  full = FALSE

-- specification AG (full -> empty) is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 2.1 <-
  data1 = 0ud1_0
  data2 = 0ud1_0
  buffer[0] = 0ud1_0
  buffer[1] = 0ud1_0
  buffer[2] = 0ud1_0
  buffer[3] = 0ud1_0
  buffer[4] = 0ud1_0
  rd_p = 0
  wr_p = 0
  empty = TRUE
  write = FALSE
  wr_data = 0ud1_0
  read = FALSE
  rd_data = 0ud1_0
  full = FALSE
-> State: 2.2 <-
  write = TRUE
-> State: 2.3 <-
  wr_p = 1
  empty = FALSE
-> State: 2.4 <-
  wr_p = 2
-> State: 2.5 <-
  wr_p = 3
-> State: 2.6 <-
  wr_p = 4
-> State: 2.7 <-
  wr_p = 0
  write = FALSE
  full = TRUE
```



```
-- specification G !(empty & full) is true
-- specification G (write -> F !empty) is true
-- specification G (((!full & wr_data = data1) & write) -> X ( G read -> (!empty U read))) is true
-- specification G ((wr_data = 0ud1_1 & write) & !full) -> ( G read -> F rd_data = 0ud1_1)) is true
-- specification G ((wr_data = data1 & write) & !full) -> ( G read -> F rd_data = data1)) is true
behzad@behzad-VMware-Virtual-Platform:~/Documents/Formal$
```

در اینجا مشخص است که تمام property های ltl درست بوده ولی ctl ها غلط میباشند یعنی در سیستم satisfy نمیشوند و در نتیجه model checker ما یک مثال نقض برای هر کدام در خروجی به ما داده است.

تعریف برخی از property ها در اینجا (خروجی بالا) عوض شده است که این اجرا مربوط به آخرین اجرا از لینوکس میباشد.